Chapter 4

MPI Programming Patterns

Collectives · uneven data · nonblocking halos · performance

What you will be able to do

- Choose a collective by where input starts and results are needed.
- Distribute uneven data and validate the result.
- Complete nonblocking exchanges safely.
- Reason about communication, synchronization, and imbalance.

From messages to patterns

Chapter 3 used explicit sends and receives. Chapter 4 groups recurring communication into reliable algorithms.

Every rank participates in a collective

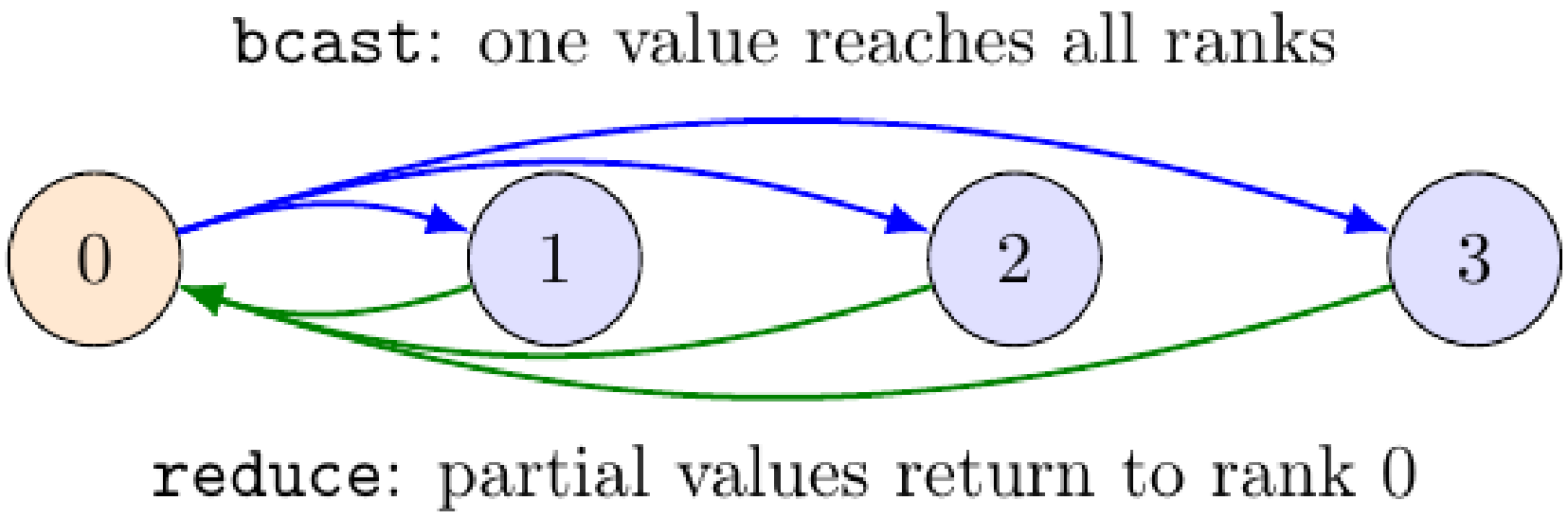
- All ranks in the communicator call compatible operations.
- They call those operations in the same order.
- The root is a rank within that communicator.
- A collective result appears only where its API specifies.

Choose by data movement

Operation	Data flow	Typical use
bcast	One root → all ranks	Share input parameters
scatter	One root → one chunk per rank	Distribute work
gather	One value per rank → root	Inspect local outputs
reduce	Local values → root	Final sum or maximum
allreduce	Local values → all ranks	Shared convergence value

Barrier coordinates arrivals; use it only when the algorithm needs a phase boundary.

Broadcast and reduction reverse the flow



The arrows show logical data flow; MPI can use a different internal network algorithm.

Only the root receives a reduce result

reduce → rank 0
allreduce → every rank

Every rank contributes a local value in both operations. Choose by who needs the answer.

One sequence on every rank

```
value = comm.bcast(value, root=0)
part = comm.scatter(chunks, root=0)
total = comm.reduce(sum(part), op=MPI.SUM, root=0)
```

If one rank skips or reorders these calls, the program may hang or fail.

Object and buffer APIs serve different data

Feature	Python objects	Typed buffers
Example	comm.scatter(chunks)	comm.Scatterv(...)
Data	Lists, scalars, dictionaries	NumPy arrays
Receive	Returned object	Preallocated compatible array
Uneven blocks	List of unequal chunks	Counts + displacements

Lower-case methods serialize Python objects; upper-case methods use typed storage.

Results appear where the operation specifies

Operation	Who receives the result?	What they receive
gather	Root only	List of rank objects
Gatherv	Root only	One typed array
reduce	Root only	One combined value
allreduce	Every rank	Same combined value

Every rank must participate even when its local block is empty.

Gatherv needs root receive storage

```
local_counts = comm.gather(count, root=0)
reconstructed = np.empty(n, dtype=np.float64) \
   if rank == 0 else None
receive_spec = [reconstructed, counts,
                displacements, MPI.DOUBLE] \
   if rank == 0 else None
comm.Gatherv(local, receive_spec, root=0)
```

Only rank 0 gets the object list and reconstructed typed array.

Reduce and allreduce differ in result location

```
total = comm.reduce(float(local.sum()),
                    op=MPI.SUM, root=0)
local_max = float(local.max()) if count else -float("inf")
global_max = comm.allreduce(local_max, op=MPI.MAX)
```

For N=7, P=4: root sum=28; every rank receives maximum=7.

A collective is not a general barrier

- Completion follows the operation’s own semantics.
- A process may leave a collective before another process does.
- Add comm.Barrier() only for a true phase boundary or careful timing.

A parallel average

The key is to combine global sums and global counts, including ranks with no elements.

An average of averages can be wrong

global mean = Σ local sums / Σ local counts

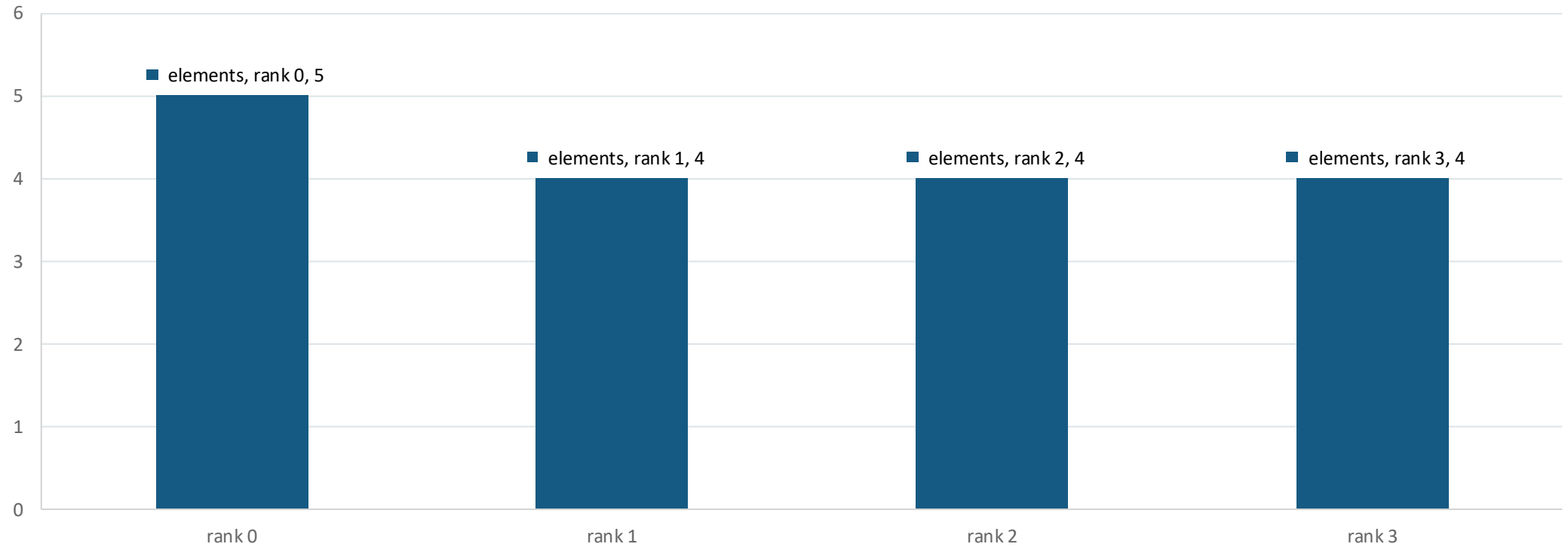
When block sizes differ, local averages carry different weights.

The 17-element example has unequal blocks

Rank	Owned values	Count	Sum	Local mean
0	1–5	5	15	3.0
1	6–9	4	30	7.5
2	10–13	4	46	11.5
3	14–17	4	62	15.5
Total	1–17	17	153	9.0

The unweighted mean of local means is 9.375; the correct global mean is 9.0.

Unequal blocks change each rank’s weight



Counts for N=17, P=4. Rank 0 contributes five values; the other ranks contribute four.

zip pairs each start with its count

Rank	Start	Count	Slice	Chunk
0	0	5	data[0:5]	1–5
1	5	4	data[5:9]	6–9
2	9	4	data[9:13]	10–13
3	13	4	data[13:17]	14–17

list(zip(starts, counts)) = [(0,5), (5,4), (9,4), (13,4)]

An empty rank still joins the collectives

Rank	Start	Count	Chunk for N=3
0	0	1	[1]
1	1	1	[2]
2	2	1	[3]
3	3	0	[]

Rank 3 receives data[3:3] = []; its local sum and count are both zero.

Broadcast N and distribute chunks

```
n = comm.bcast(n, root=0)
q, remainder = divmod(n, size)
counts = [q + (r < remainder) for r in range(size)]
starts = [r*q + min(r, remainder) for r in range(size)]
chunks = [data[s:s+c] for s, c in zip(starts, counts)]
local = comm.scatter(chunks, root=0)
```

The code shown here is from collective_average.py.

Reduce two values, then divide

```
local_sum = sum(local)
local_count = len(local)
global_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
global_count = comm.reduce(local_count, op=MPI.SUM, root=0)
if rank == 0:
    average = global_sum / global_count
```

An empty local list contributes sum=0 and count=0.

Check the mathematical answer

$$1 + 2 + \cdots + 17 = 153$$

$$153 / 17 = 9$$

The root also compares the parallel answer with a serial calculation.

Run cases that expose boundary errors

N	Ranks	Why run it	Expected mean
17	4	Uneven blocks	9.0
3	4	One empty block	2.0
17	1	Single-process baseline	9.0

```
mpiexec -n 4 python -m mpi4py collective_average.py 17
```

The simple scatter has a cost

- Rank 0 constructs the full data and the chunk list.
- That centralizes memory use and distribution.
- It is a clear teaching example, but large arrays need a more suitable layout.

Uneven numerical arrays

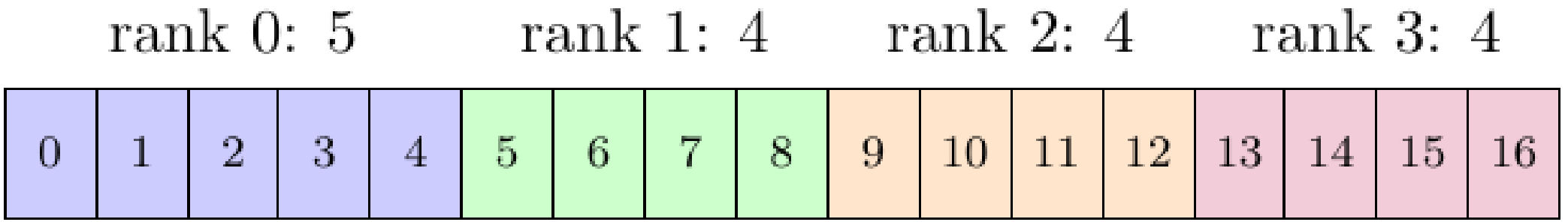
Scatterv distributes typed array blocks with counts and element displacements.

Counts and displacements define the blocks

Rank	Count	Displacement	Half-open range
0	5	0	[0, 5)
1	4	5	[5, 9)
2	4	9	[9, 13)
3	4	13	[13, 17)

Displacements count MPI.DOUBLE elements here, not bytes.

Seventeen indices split across four ranks



Colors identify the four contiguous blocks; the final three ranks each receive four elements.

The offset formula works for every rank

$$count_r = q + [r < remainder]$$
$$start_r = r \cdot q + \min(r, remainder)$$

For N=17 and P=4, q=4 and remainder=1.

Build counts and displacements

```
q, remainder = divmod(n, size)
counts = np.array([q + (r < remainder)
                   for r in range(size)], dtype=np.int32)
displacements = np.zeros(size, dtype=np.int32)
displacements[1:] = np.cumsum(counts[:-1])
```

For N=17, P=4: counts=[5,4,4,4], displacements=[0,5,9,13].

Allocate exactly the local receive capacity

```
local_n = int(counts[rank])
local_x = np.empty(local_n, dtype=np.float64)
local_y = np.empty(local_n, dtype=np.float64)
comm.Scatterv([x, counts, displacements, MPI.DOUBLE],
        local_x, root=0)
comm.Scatterv([y, counts, displacements, MPI.DOUBLE],
        local_y, root=0)
```

The MPI datatype and NumPy dtype must be compatible.

Each rank computes one partial dot product

$$d_r = \Sigma\, x_i y_i \text{ over its block}$$

$$x \cdot y = \Sigma\, d_r$$

Reduce the partial products to rank 0 for the final answer.

Compute, reduce, and validate

```
local_dot = float(np.dot(local_x, local_y))
global_dot = comm.reduce(local_dot, op=MPI.SUM, root=0)
if rank == 0:
    expected = float(np.dot(x, y))
    passed = np.isclose(global_dot, expected)
```

For x=1…17 and y=2, the expected result is 2×153=306.

Floating results need a sensible check

Case	Expected	Important behavior
N=17, P=4	306	Uneven blocks
N=3, P=4	12	One zero-length receive array
N=0, P=4	0	All ranks still enter collectives

Use np.isclose because reduction order can change floating-point rounding.

Nonblocking halo exchange

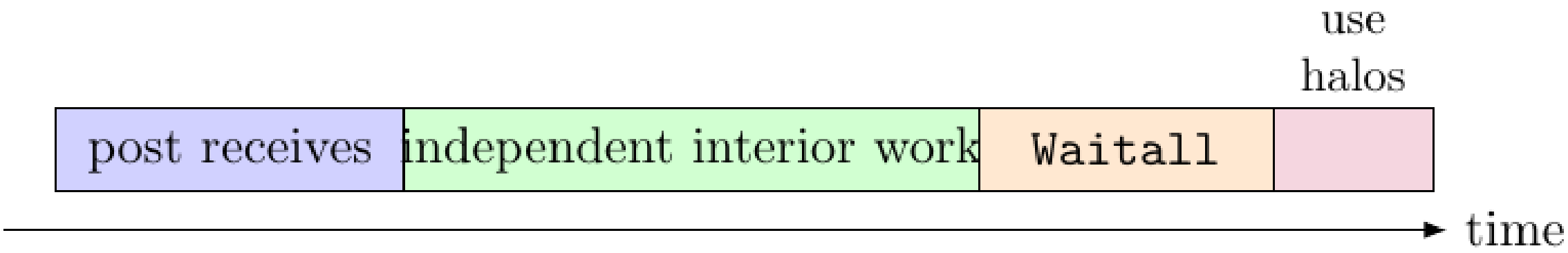
Ranks share boundary values while respecting request and buffer lifetimes.

Posting is not completion

Irecv / Isend → Request
Waitall → completed data

The returned request must be kept until the operation completes.

Overlap is an opportunity, not a promise



Post communication, do independent work, wait, then use received halos. This timeline is conceptual.

Active buffers have ownership rules

State	Safe rule
Active Isend	Keep array alive; do not modify sent region
Active Irecv	Do not read or write receive region
After Waitall	Reuse sends; read completed receives

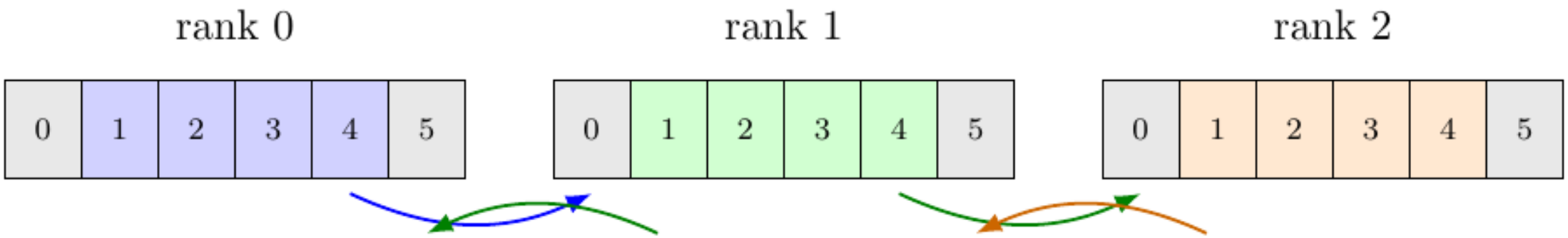
A call returning from Isend or Irecv does not mean the transfer is complete.

Ghost cells hold a neighbor’s boundary

[ghost | owned cells 1–4 | ghost]

A rank computes on its owned cells while ghost cells provide values at the edge.

Boundary values move into ghost cells



Three ranks are shown. Gray cells are ghosts; colored cells belong to each rank.

The tags describe data direction

Direction	Send	Matching receive
Moving right	send right, tag 20	receive from left, tag 20
Moving left	send left, tag 21	receive from right, tag 21

A reversed receive tag can leave a halo stale or hang the exchange.

Post receives before sends

```
requests = [
    comm.Irecv(field[0:1], source=left, tag=20),
    comm.Irecv(field[5:6], source=right, tag=21),
    comm.Isend(field[1:2], dest=left, tag=21),
    comm.Isend(field[4:5], dest=right, tag=20),
]
```

The slices are one-element NumPy buffers.

Do independent work, then wait

```
interior_total = float(np.sum(field[2:4]))
MPI.Request.Waitall(requests)
# Now field[0] and field[5] may be read.
print(field.tolist())
```

The interior calculation does not touch active send or receive regions.

PROC_NULL handles a missing neighbor

left = rank - 1 if rank > 0 else MPI.PROC_NULL

A send or receive with MPI.PROC_NULL is a safe no-op at the physical boundary.

Predict the four-rank fields

Rank	Left ghost	Owned values	Right ghost
0	−1	0, 0, 0, 0	1
1	0	1, 1, 1, 1	2
2	1	2, 2, 2, 2	3
3	2	3, 3, 3, 3	−1

The −1 sentinels illustrate missing neighbors; a simulation needs physical boundary conditions.

One heat step spreads an impulse

Cell index	0	1	2	3	4
Old value	0	0	1	0	0
New value	0	0.25	0.50	0.25	0

Using λ=0.25 and fixed-zero physical boundaries. Every new cell reads old values.

A one-cell block has no independent interior

- Post halo receives and sends before updating interior cells.
- Waitall before reading a received ghost or changing a send buffer.
- Update local edge cells after the wait; update a one-cell block once.
- Reset a physical-boundary ghost to zero every step.

Display barriers are separate from the algorithm

- The halo messages complete at Waitall.
- The final loop’s barriers only print ranks in order.
- A barrier in every short compute step can add waiting.

Model the costs before measuring

Latency, transferred bytes, synchronization, and uneven work all influence runtime.

A message pays startup and transfer cost

$$T(n) = \alpha + \beta n$$

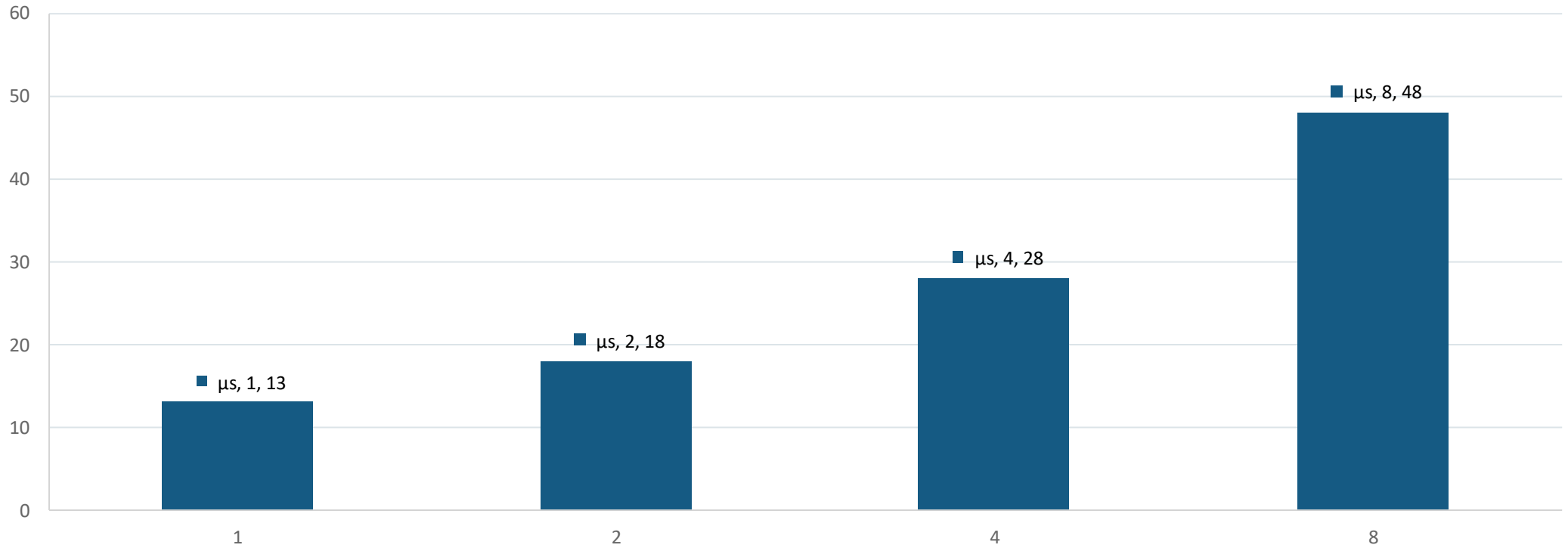
α is latency; β is time per byte. The simple model omits contention, copies, and system effects.

Same payload, different message counts

Messages	Bytes/message	Total bytes	Model time (µs)
1	8,000	8,000	13
2	4,000	8,000	18
4	2,000	8,000	28
8	1,000	8,000	48

Calculated using α=5 µs and β=1 ns/byte; these are not measurements.

Startup cost grows with message count



Analytical model for 8,000 total bytes. More messages repeat the 5 µs startup cost.

Time fixed work and report the slowest rank

```
comm.Barrier()
t0 = MPI.Wtime()
# Run the same fixed number of steps on every rank.
local_elapsed = MPI.Wtime() - t0
phase_time = comm.reduce(local_elapsed,
                         op=MPI.MAX, root=0)
```

Exclude setup and printing; repeat runs and report the median maximum time.

Record measured scaling separately

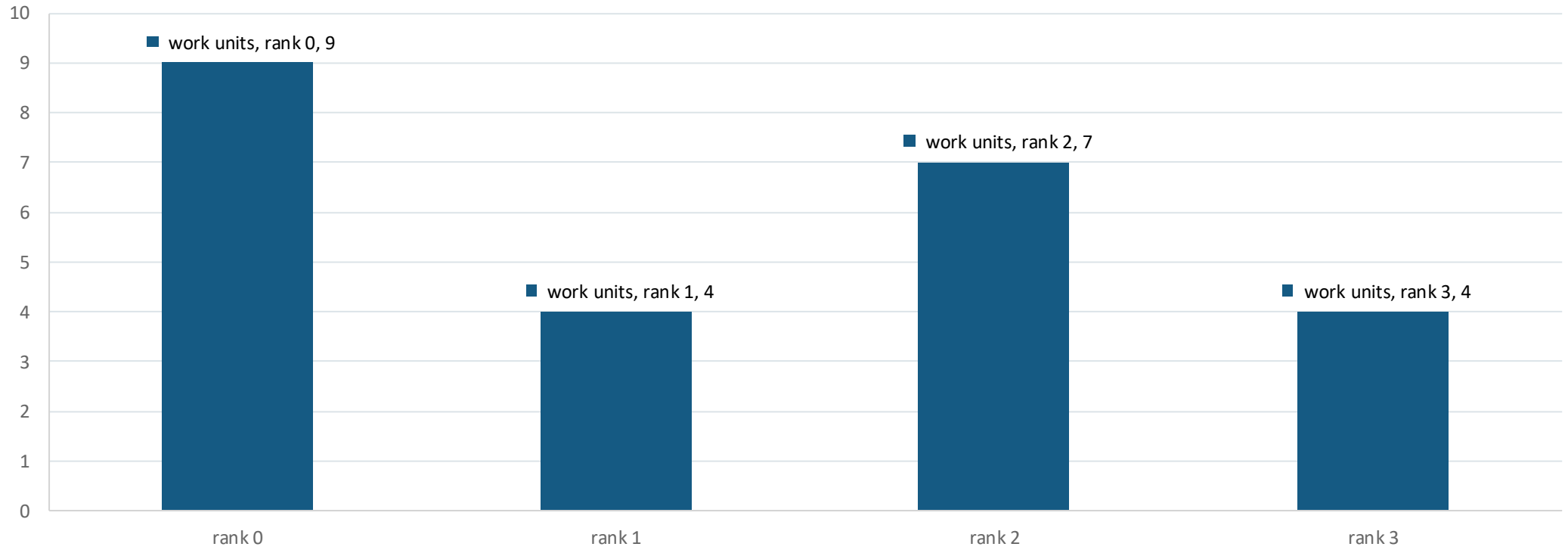
Ranks	Median time T(P)	Speedup T(1)/T(P)	Efficiency S(P)/P
1	measure	1.00	1.00
2	measure	calculate	calculate
4	measure	calculate	calculate

Use one fixed problem size and step count; document the machine and rank placement.

Batching needs application measurement

- Fewer messages can reduce repeated startup overhead.
- Waiting to fill a batch may delay dependent work.
- Compare repeated timings on specified hardware before claiming speedup.

Equal task counts can hide unequal work



Illustrative work units with four tasks per rank. The phase waits for rank 0's nine units.

Choose a distribution for the workload

Approach	Useful when	Tradeoff
Contiguous blocks	Uniform work per element	Poor if costs cluster
Cyclic assignment	Costs vary by position	Less locality
Small task queue	Costs are unpredictable	Scheduling overhead

A work-count chart does not establish actual runtime speedup.

A barrier waits for the slowest arrival

- Use a barrier for a required global phase boundary.
- Neighbor exchange or a reduction may be enough for other phases.
- Keep collective order compatible on every branch.

Start with the symptom

Symptom	Likely cause	First check
Collective hangs	Different call order	Trace each rank’s sequence
Wrong average	Unweighted rank means	Reduce sum and count
Wrong dot product	Counts, offsets, dtype	Print local ranges
Stale halo	Tag, neighbor, wait	Trace request completion

Prefix debug lines with rank and flush; print order alone does not show computation order.

Use tests that expose layout mistakes

Program	Single rank	Uneven	Empty partition
Average	N=17, P=1	N=17, P=4	N=3, P=4
Dot product	N=17, P=1	N=17, P=4	N=0 or 3, P=4
Halo	P=1	P=4	Boundary ghosts

Check serial and mathematical results where available.

Apply the patterns

Use the examples as small experiments and explain both the result and the edge cases.

Run and explain the average

- Run N=17 with one, two, and four ranks.
- Record global sum, count, and mean.
- Run N=3 with four ranks and identify the empty rank.
- Explain why rank averages must be weighted.

Inspect where collective results appear

- Run collective_results.py with N=7, P=4 and N=3, P=4.
- Record the root results from gather, Gatherv, and reduce.
- Check that every rank gets the same allreduce maximum.

Extend the dot product and halo

- Compute counts and displacements for N=23, P=5.
- Run the dot product with N=0 and N=3, P=4.
- Run halos with two and four ranks; predict the ghosts.
- Change each halo rank from four owned values to eight.

GUIDED LAB

Measure scaling with fixed work

- Time the same problem size and step count at P=1, 2, and 4.
- Report the median of at least three maximum rank times.
- Calculate speedup and efficiency from measured times.

Keep four rules in mind

- Every rank enters compatible collectives in the same order.
- Uneven blocks require counts and element offsets.
- Nonblocking buffers remain protected until completion.
- Validate correctness before interpreting performance.

The pattern is the program

distribute → compute locally → communicate → validate

Use the right collective or neighbor exchange for each stage, then test one rank, uneven work, and empty partitions.